Sprawozdanie z projektu nr 1

Implementacja gry zręcznościowej Arkanoid w środowisku Python

Kazimierz Szyszkowski
Ewa Dziś

22 marca 2026

Spis treści

1 Wstęp i cel projektu **2**

2 Analiza wymagań i założenia **2**
2.1 Założenia funkcjonalne 2
2.2 Specyfikacja parametrów gry 2

3 Opis architektury i rozwiązań programowych **2**
3.1 Algorytm detekcji kolizji 2
3.2 Zarządzanie czasem i pętlą gry 3

4 Dyskusja wyników i testy **3**
4.1 Analiza wydajności 3
4.2 Wady i propozycje ulepszeń 3

5 Wnioski końcowe **3**

1 Wstęp i cel projektu

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie interaktywnej gry typu Arkanoid. Głównym wyzwaniem programistycznym było stworzenie wydajnego silnika wykrywania kolizji w czasie rzeczywistym oraz obsługa zdarzeń wejściowych (klawiatura/mysz) w asynchronicznej pętli graficznej.

2 Analiza wymagań i założenia

2.1 Założenia funkcjonalne

- **Dynamiczny ruch obiektu:** Piłka porusza się ze stałą prędkością wektorową $[dx, dy]$.
- **Interakcja z użytkownikiem:** Sterowanie paletką ograniczone do osi X, z zachowaniem granic okna gry.
- **Mechanika zniszczeń:** Każdy blok posiada atrybut widoczności, który po kolizji zmienia stan na *False*.
- **Warunki końcowe:** Wygrana następuje po wyczyszczeniu listy bloków, przegrana po kontakcie piłki z dolną krawędzią (Y > Y_max).

2.2 Specyfikacja parametrów gry

Parametr	**Wartość/Opis**
Język programowania	Python 3.x
Biblioteka GUI	Tkinter / Pygame
Odświeżanie (FPS)	ok. 60 Hz (interwał 15-17ms)
Wymiary okna	800x600 px

3 Opis architektury i rozwiązań programowych

Aplikacja została zaprojektowana w paradygmacie programowania obiektowego (OOP). Główna klasa `ArkanoidGame` zarządza stanem gry.

3.1 Algorytm detekcji kolizji

Zastosowano algorytm sprawdzania przecięcia prostokątów ograniczających (AABB - *Axis-Aligned Bounding Box*).

```python
def update_ball(self):
    self.canvas.move(self.ball, self.dx, self.dy)
    pos = self.canvas.coords(self.ball)

    # Kolizja z blokami
    for brick in self.bricks[:]:
        if self.canvas.find_overlapping(*self.canvas.coords(brick)):
            self.dy = -self.dy
            self.canvas.delete(brick)
            self.bricks.remove(brick)
            self.score += 10
```

Listing 1: Rozbudowana logika odbić i usuwania bloków

3.2 Zarządzanie czasem i pętlą gry

Zamiast nieskończonej pętli `while`, wykorzystano metodę `.after()` biblioteki Tkinter, co zapewnia nieblokujący charakter interfejsu użytkownika i pozwala systemowi operacyjnemu na obsługę innych zdarzeń.

4 Dyskusja wyników i testy

4.1 Analiza wydajności

Podczas testów aplikacja wykazała stałe zużycie zasobów procesora na poziomie poniżej 5%. Opóźnienia sterowania (input lag) są niezauważalne dla użytkownika.

4.2 Wady i propozycje ulepszeń

1. **Błąd "tunelowania":** Przy bardzo dużych prędkościach piłka może przejść przez cienki obiekt. Rozwiązaniem byłoby zastosowanie ciągłej detekcji kolizji (CCD).

2. **Brak urozmaiceń:** Wersja podstawowa nie zawiera "Power-upów", co można zaimplementować dodając losowanie bonusów po zniszczeniu bloku.

5 Wnioski końcowe

Projekt pozwolił na zrozumienie fundamentalnych zasad tworzenia silników gier 2D. Kluczowym wnioskiem jest konieczność precyzyjnego zarządzania układem współrzędnych, gdzie punkt $(0, 0)$ znajduje się w lewym górnym rogu ekranu, co wymaga odwrócenia intuicyjnego myślenia o osi Y.